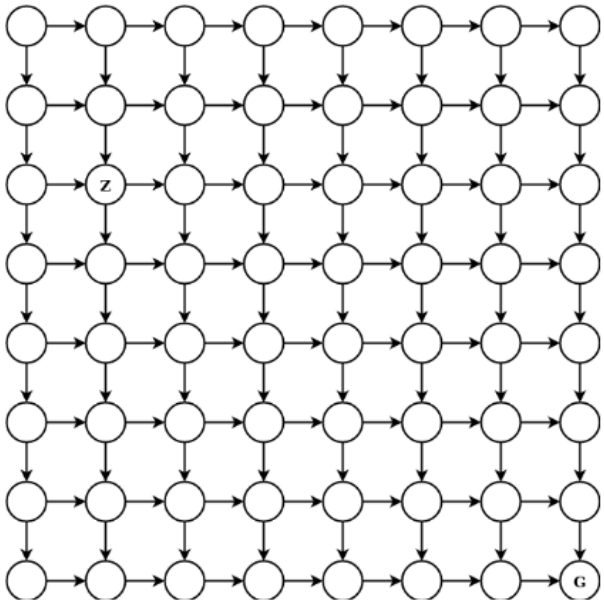
National University of Singapore
School of Computing
CS3244: Machine Learning
Solution to Tutorial 06

**Reinforcement Learning**

1. **Markov Decision Process.** Describe a task that fits into the MDP framework, then identify its states, actions, and rewards.

   *We can model an agent that is playing Mario as an MDP. The states are the frames, the actions are up, down, left, right, jump and shoot, and the reward is the score.*

2. **Value Iteration.** The following directed acyclic graph represents a MDP with $n^2$ states where $n = 8$. Each vertex of the graph represents a state, and each outgoing arrow from state $X$ represents an action that can be taken in $X$. A special action must be taken in goal state $G$ to terminate the game. All actions taken will incur a reward of 1 (including the special action taken in $G$). Take $\gamma = 0.9$.




   (a) How many iterations does it take for the $V(Z)$ to converged to $V^*(Z)$?

   *This question does not require any brute-forcing. The number of iterations required for any state $S$ to converge to $V*(S)$ is one more than the longest path from $S$ to $G$. Thus, we need 12 iterations here.*

   (b) Fixing $Z$ on position $(3, 2)$ and $G$ on the position $(n, n)$, what is $V^*(Z)$ as $n \to \infty$?

   *Using geometric series, we get $V^*(Z) = 1 + 0.9 + 0.9^2 + \cdots = 1/(1 - 0.9) = 10$.*

3. **Reinforcement Learning.** Recall that in reinforcement learning, we do not have access to the transition function $p(s', r \mid s, a)$ and reward function $R(s, a)$. As a result, dynamic programming methods (such as value iteration) are no longer feasible.

   (a) In this question, we present two algorithms that allow learning when the transition and reward functions are unknown. Give an intuitive explanation on how these algorithm works.

**Monte Carlo ES (Exploring Starts), for estimating $\pi \approx \pi_*$**

Initialize:
  $\pi(s) \in \mathcal{A}(s)$ (arbitrarily), for all $s \in \mathcal{S}$
  $Q(s, a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$
  $Returns(s, a) \leftarrow$ empty list, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

Loop forever (for each episode):
  Choose $S_0 \in \mathcal{S}$, $A_0 \in \mathcal{A}(S_0)$ randomly such that all pairs have probability $> 0$
  Generate an episode from $S_0, A_0$, following $\pi$: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$
  $G \leftarrow 0$
  Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:
    $G \leftarrow \gamma G + R_{t+1}$
    Unless the pair $S_t, A_t$ appears in $S_0, A_0, S_1, A_1 \ldots, S_{t-1}, A_{t-1}$:
      Append $G$ to $Returns(S_t, A_t)$
      $Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$
      $\pi(S_t) \leftarrow \arg\max_a Q(S_t, a)$

*Instead of solving for $Q(s, a)$, we can estimate it by running through a few episodes. Using the estimate $\hat{Q}(s, a)$, we can greedily update our policy.*

**Sarsa (on-policy TD control) for estimating $Q \approx q_*$**

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(terminal, \cdot) = 0$

Loop for each episode:
  Initialize $S$
  Choose $A$ from $S$ using policy derived from $Q$ (e.g., $\varepsilon$-greedy)
  Loop for each step of episode:
    Take action $A$, observe $R$, $S'$
    Choose $A'$ from $S'$ using policy derived from $Q$ (e.g., $\varepsilon$-greedy)
    $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma Q(S', A') - Q(S, A)\big]$
    $S \leftarrow S'$; $A \leftarrow A'$;
  until $S$ is terminal

*This works similarly to Monte Carlo ES. Intuitively, it make sense that our estimate of the expected return $\hat{Q}(s_t, a_t)$ would be better if we take into account the reward we obtained after taking one more step. In other words, the estimator $r_{t+1} + \gamma\hat{Q}(s_{t+1}, a_{t+1})$ is a better estimate of the true $Q(s_t, a_t)$ than $\hat{Q}(s_t, a_t)$.*

   (b) Explain why you cannot run Monte Carlo ES when the number of time steps in each episode is possibly infinite.

*We can only obtain the estimated $\hat{Q}(s, a)$ after every episode.*